

Съхранени процедури, входни параметри, глобални и локални променливи в MySQL

Съхранените процедури (Stored Procedures) представляват група от SQL заявки, съхранени в базата данни, които могат да бъдат изпълнявани многократно с различни параметри. Те наподобяват функциите в програмирането и позволяват автоматизация на често използвани операции, като например:

- Генериране на отчети и справки
- Проверки на данни
- Прехвърляне на информация между таблици
- Извършване на последователност от операции, например приключване на отчетен период

Използването на съхранени процедури намалява трафика между приложението и базата данни, тъй като множество заявки могат да бъдат изпълнени в една процедура. Те също така подобряват производителността, като позволяват кеширане на компилирания код от сървъра.

Създаване на съхранена процедура в MySQL

В MySQL, процедурите се дефинират със следния синтаксис:

```
DELIMITER //
```

```
CREATE PROCEDURE procedure_name()
```

```
BEGIN
```

```
    -- SQL заявки
```

```
END //
```

```
DELIMITER ;
```

Забележка:

- DELIMITER се използва за промяна на символа за край на заявка, за да позволи многоредови процедури.
- Вътре в BEGIN ... END се поставят SQL инструкциите, които ще бъдат изпълнени при извикване на процедурата.

Пример:

Създаване на процедура, която извлича всички групи и свързаните с тях спортове:

```
delimiter //  
  
CREATE PROCEDURE getAllSportGroupsWithSports()  
  
BEGIN  
  
    SELECT sg.location AS locationOfGroup,  
           sg.dayOfWeek AS trainingDay,  
           sg.hourOfTraining AS trainingHour,  
           sp.name AS sportName  
  
    FROM sportgroups sg  
  
    JOIN sports sp ON sg.sport_id = sp.id;  
  
END //  
  
delimiter ;
```

За да извикаме процедурата, използваме:

```
CALL getAllSportGroupsWithSports();
```

Променливи в MySQL

Глобални променливи

Глобалните променливи в MySQL се обозначават с @ и са валидни само за текущата сесия. Те могат да бъдат от следните типове: INTEGER, DECIMAL, FLOAT, STRING или NULL.

Пример:

```
SET @customer_number = 'A454647';  
  
SELECT @customer_number;
```

Глобалните променливи могат да се използват и в заявки:

```
SET @coach_name = 'Иван Тодоров Петров';  
  
SELECT * FROM coaches WHERE name = @coach_name;
```

Локални променливи

Локалните променливи се използват в рамките на съхранени процедури и се декларират с DECLARE.

Пример:

```
DELIMITER //  
CREATE PROCEDURE exampleProc()  
BEGIN  
    DECLARE coach_name VARCHAR(100);  
    SET coach_name = 'Иван Петров';  
    SELECT coach_name;  
END //  
DELIMITER ;
```

Локалните променливи съществуват само докато процедурата се изпълнява и не могат да бъдат използвани извън нея.

Входни параметри за процедури

В зависимост от начина на използване, параметрите в процедурите могат да бъдат три вида:

1. **IN** – Предава се стойност към процедурата (аналогично на аргументите по стойност в програмирането). **Предоставят стойност на процедурата.**
2. **OUT** – Процедурата задава стойност на параметъра, която може да се използва след изпълнението. **Връщат стойност от процедурата.**
3. **INOUT** – Комбинира поведението на IN и OUT – параметърът може да бъде подаден с начална стойност, да бъде променен вътре в процедурата и резултатът да бъде използван след това. **Приемат и връщат стойност.**

IN параметър

```
DELIMITER //  
  
CREATE PROCEDURE getSportGroupsBySport(IN sportName VARCHAR(50))  
  
BEGIN  
  
    SELECT sg.location, sg.dayOfWeek, sg.hourOfTraining  
  
    FROM sportgroups sg  
  
    JOIN sports sp ON sg.sport_id = sp.id  
  
    WHERE sp.name = sportName;  
  
END //  
  
DELIMITER ;
```

Извикване на процедурата:

```
CALL getSportGroupsBySport('Football');
```

OUT параметър

OUT параметрите позволяват процедурата да върне стойност.

Тази процедура връща броя на групите за даден спорт.

```
DELIMITER //  
  
CREATE PROCEDURE countSportGroupsBySport(IN sportName VARCHAR(50), OUT totalGroups INT)  
  
BEGIN  
  
    SELECT COUNT(*) INTO totalGroups  
  
    FROM sportgroups sg  
  
    JOIN sports sp ON sg.sport_id = sp.id  
  
    WHERE sp.name = sportName;  
  
END //  
  
DELIMITER ;
```

Извикване на процедурата:

```
CALL countSportGroupsBySport('Basketball', @groupCount);  
  
SELECT @groupCount;
```

INOUT параметър

INOUT параметърът може да се подаде с начална стойност, да бъде променен в процедурата и след това да се използва.

```
DELIMITER //
```

```
CREATE PROCEDURE increaseCoachSalary(IN coachID INT, INOUT salaryIncrease DOUBLE)
```

```
BEGIN
```

```
    UPDATE coaches
```

```
    SET salary = salary + salaryIncrease
```

```
    WHERE id = coachID;
```

```
-- Връщаме новата заплата
```

```
    SELECT salary INTO salaryIncrease FROM coaches WHERE id = coachID;
```

```
END //
```

```
DELIMITER ;
```

Извикване на процедурата:

```
SET @newSalary = 500;
```

```
CALL increaseCoachSalary(1, @newSalary);
```

```
SELECT @newSalary;
```

Важно:

- При OUT и INOUT параметрите трябва да се подават съществуващи променливи.
- CALL inoutParamProc('Ivan Petrov'); ще доведе до грешка, защото не подава валидна променлива.

Съхранените процедури са мощен инструмент за оптимизация на SQL операциите. Те позволяват:

- Изпълнение на сложни заявки с минимален трафик към сървъра
- Използване на параметризирани заявки за динамични условия
- Намаляване на дублирането на SQL код в приложенията

Използването на IN, OUT и INOUT параметри прави процедурите гъвкави и подходящи за различни сценарии. Локалните и глобалните променливи улесняват управлението на данните в рамките на сесията и процедурите.